4.1 Namespaces and name resolution

Two files can have different paths and still declare the same PHP class. Loading two global classes named User produces a naming collision. Namespaces give those classes distinct names; loading their files is a separate job.

Declare the first class

Create `src/Tutorials/Class4/User.php`:

```php
<?php
declare(strict_types=1);

namespace App\Tutorials\Class4;

class User
{
    public function __construct(private string $name)
    {
    }

    public function getName(): string
    {
        return $this->name;
    }
}
```

The class's full name is `App\Tutorials\Class4\User`. The namespace declaration comes before ordinary executable statements; a strict-types declaration may precede it. A namespace is a naming system, not a command to create or search directories.

Create `public/tutorials/4-1-namespaces.php`:

```php
<?php
declare(strict_types=1);

use App\Tutorials\Class4\User;

require_once __DIR__ . '/../../src/Tutorials/Class4/User.php';

$user = new User('Avery');
echo $user->getName(), PHP_EOL;
```

Run `php public/tutorials/4-1-namespaces.php` from the repository root. Expect `Avery`.

Temporarily remove the `use` line and change construction to `new \App\Tutorials\Class4\User('Avery')`. It still works: the leading backslash makes the reference fully qualified. Restore the import for readability. Now temporarily remove only the require line: construction fails because `use` does not load a file. Restore it.

Resolve a collision with an alias

Create `src/Tutorials/Class4/Admin/User.php`:

```php
<?php
declare(strict_types=1);

namespace App\Tutorials\Class4\Admin;

class User
{
    public function __construct(private string $name)
    {
    }

    public function getName(): string
    {
        return 'Administrator: ' . $this->name;
    }
}
```

Replace `public/tutorials/4-1-namespaces.php` with:

```php
<?php
declare(strict_types=1);

use App\Tutorials\Class4\Admin\User as AdminUser;
use App\Tutorials\Class4\User;

require_once __DIR__ . '/../../src/Tutorials/Class4/User.php';
require_once __DIR__ . '/../../src/Tutorials/Class4/Admin/User.php';

echo (new User('Avery'))->getName(), '<br>', PHP_EOL;
echo (new AdminUser('Morgan'))->getName(), '<br>', PHP_EOL;
```

Expect `Avery` and `Administrator: Morgan` on separate lines. An alias changes the local spelling in this file; it does not rename the original class. Imports do not carry over into other files.

Explain name resolution inside a namespace

Within `namespace App\Tutorials\Class4;`, an unimported `User` means `App\Tutorials\Class4\User`. An unimported `Tools\Formatter` means `App\Tutorials\Class4\Tools\Formatter`. `\DateTimeImmutable` explicitly names the built-in class. Prefer `use DateTimeImmutable;` at the top when using it repeatedly. Class names do not fall back to the global namespace when no matching class exists.

Namespaces also support functions and constants, imported with `use function` and `use const`. These imports still do not load declaration files. Our class autoloading lessons will not autoload standalone functions on demand.

Reference: PHP name resolution rules.